Introducing Instruction-Accurate Simulators for Performance Estimation of Autotuning Workloads

Rebecca Pelke, Nils Bosbach, Lennart M. Reimann, Rainer Leupers
Institute for Communication Technologies and Embedded Systems
RWTH Aachen University, Germany
{pelke, bosbach, lennart.reimann, leupers}@ice.rwth-aachen.de

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133237

***Abstract*—Accelerating Machine Learning (ML) workloads requires efficient methods due to their large optimization space. Autotuning has emerged as an effective approach for systematically evaluating variations of implementations. Traditionally, autotuning requires the workloads to be executed on the target hardware (HW). We present an interface that allows executing autotuning workloads on simulators. This approach offers high scalability when the availability of the target HW is limited, as many simulations can be run in parallel on any accessible HW.**

Additionally, we evaluate the feasibility of using fast instruction-accurate simulators for autotuning. We train various predictors to forecast the performance of ML workload implementations on the target HW based on simulation statistics.

Our results demonstrate that the tuned predictors are highly effective. The best workload implementation in terms of actual run time on the target HW is always within the top 3 % of predictions for the tested x86, ARM, and RISC-V-based architectures. In the best case, this approach outperforms native execution on the target HW for embedded architectures when running as few as three samples on three simulators in parallel.

***Index Terms*—Autotuning, TVM, gem5, cache optimization**

I. Introduction

The optimization of Machine Learning (ML) models is crucial due to their high computational demands. Traditional analytical methods often fall short given the huge search space for optimal implementations on modern CPU architectures. To address this challenge, autotuning has emerged as a powerful approach. Autotuning systematically evaluates multiple implementations of the same workload using mathematical models or ML techniques to guide subsequent selections of implementations [1], [2]. Apache Tensor Virtual Machine (TVM) [3], a well-known ML compiler framework, implements several autotuning strategies [4], [5]. Autotuning typically requires execution on real hardware, which introduces non-determinism due to factors such as the system load [6], cache collisions [7], thermal throttling [8], and frequency and voltage scaling [9].

To mitigate these issues, each benchmark is executed multiple times, outliers are removed, cooldown periods are inserted, and caches are flushed before each repetition. Consequently, benchmarking a single implementation takes significantly longer than its actual run time. This is time-consuming, especially when only limited hardware (HW) devices are available. This paper presents the following two main contributions:

Contribution Ⓘ: Simulator Interface - We present an interface allowing autotuning workloads to be executed on simulators rather than real hardware. Figure 1 Ⓘ illustrates this approach. We extract the TVM tasks and provide them as executables for the target architecture through a simulator interface. The number of parallel tasks is configurable, each running in a separate instance of the simulator. Potential scenarios that profit from our **simulator interface** are:

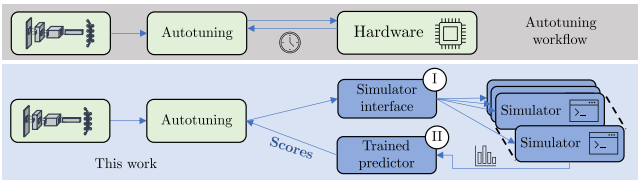


Fig. 1: The proposed simulator interface Ⓘ and the score predictor approach using instruction-accurate simulators Ⓘ

- The HW is not yet available, e.g., for pre-silicon software (SW) development.
- The embedded HW is only available in limited quantities, making parallel execution of simulations on high-performance computers faster than native execution.
- Other metrics besides run time should be optimized.

Contribution Ⓘ: Score Predictor - We aim to demonstrate that instruction-accurate simulators can be used for performance analysis, using autotuning workloads as an example. *To the best of our knowledge, we are the first to show how performance analysis can be conducted with fast instruction-accurate simulators.* Many open-source implementations of instruction-accurate simulators exist for different architectures, e.g., QEMU [10] or gem5 [11]. Figure 1 Ⓘ illustrates this idea. A predictor gets statistics from an instruction-accurate simulator as input and calculates a score based on reference values measured on real hardware. The score is then returned to the autotuning framework.

Since an instruction-accurate simulator does not provide accurate timing, we do **not** aim to predict execution latencies. Instead, our approach utilizes scores to evaluate and compare different implementations of the same workload. These scores are essential for guiding the autotuning process but are not suitable for comparing different types of workloads. We employ multiple predictors for this task: Multiple Linear Regression (MLR) [12], Deep Neural Networks (DNNs) [13], Bayesian optimization [14], and XGBoost [15]. We tune and compare them to identify the most accurate predictions for common ML kernels. We evaluate our approach on different CPU architectures, namely x86, ARM, and RISC-V.

II. Background and Related Work

A. Autotuning in TVM

Apache TVM [3] is an open-source ML compiler framework designed to optimize computations across various hardware platforms. In TVM, each operation, called *kernel* in this work, can be expressed in different abstractions, e.g., in Tensor Expression (TE) or in Tensor Intermediate Representation (TIR). For example, a kernel can be a Neural Network (NN) layer. TVM enables the application of *transformation primitives* to optimize kernels for diverse target architectures.

Similar to Halide [16], TVM distinguishes between the *compute* operation, which defines the functional behavior of the kernel, and the *schedule*, which defines its implementation. This distinction allows a single computation to have multiple schedules. The set of all possible schedules of an operation is called *design space*. Large design spaces and complex hardware behavior make analytical approaches impractical for finding optimal schedules for ML kernels. Autotuning addresses this issues by empirically evaluating multiple schedules directly on the target hardware. TVM offers three autotuning concepts: the AutoTVM framework, the Auto-Scheduler (also called Ansor [4]), and the Meta-Scheduler. This work focuses on AutoTVM and the Auto-Scheduler since they operate on the same input representation, namely TE.

Listing 1 provides an example of a TE compute operation definition. Tensor C (size $N \times M$) contains the result of a Matrix Matrix Multiplication (MMM) between matrices A (size $N \times L$) and B (size $L \times M$).

```
k = te.reduce_axis((0, L), name="k")
C = te.compute((N, M), lambda i, j: \
    te.sum(A[i,k]*B[k,j], axis=k), name="matmul")
```

Listing 1: Definition of a MMM compute operation in TE

AutoTVM requires users to define templates with tunable parameters, such as loop tiling factors. AutoTVM then navigates this search space by executing configurations on real hardware and measuring the run time. It requires some expertise to define a useful schedule template. However, predesigned templates for common operators can be found in the repository. As an example, Listing 2 illustrates how a *split* primitive of a single axis can be described using AutoTVM.

```
s = te.create_schedule(C.op)
y,_ = s[C].op.axis
# Define schedule template
cfg = autotvm.get_config()
cfg.define_split("split_y", y, num_outputs=2,...)
# Apply schedule
yo, yi = cfg["split_y"].apply(s, C, y)
```

Listing 2: Definition of a scheduling template for AutoTVM

In contrast to AutoTVM, the Auto-Scheduler automates the schedule generation process without requiring manual template definitions, making it more suitable for non-experts.

Figure 2 illustrates the workflows of both the Auto-Scheduler and AutoTVM. Unlike the manually defined search space in AutoTVM, the Auto-Scheduler uses *sketches* generated from the kernel's Directed Acyclic Graph (DAG) through

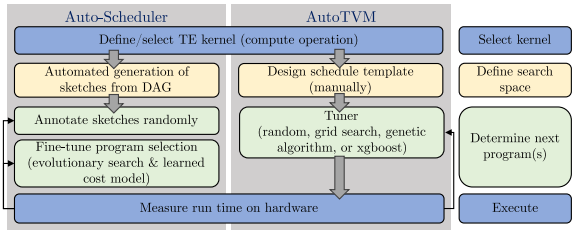


Fig. 2: Autotuning using Auto-Scheduler and AutoTVM

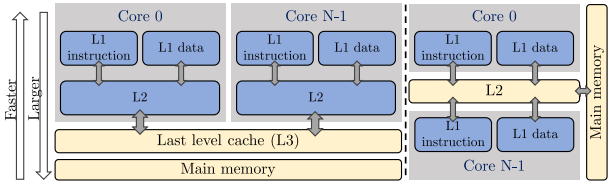


Fig. 3: Typical cache hierarchies of modern CPUs

predefined derivation rules. A sketch basically contains nested loops with placeholders, which are filled during an *annotation* phase. During annotation, loop axes can also be marked for parallelization, unrolling, or vectorization. Implementation details can be found in [4]. In contrast to the Auto-Scheduler, AutoTVM relies on tuners responsible for selecting subsequent programs based on selectable tuning algorithms.

B. Cache Hierarchy

ML operations require cache optimizations to achieve high performance [17], [18]. Modern CPUs have hierarchical memory systems with multiple cache levels (L1, L2, L3) as shown in Figure 3. The hierarchy can vary between different CPUs [19]. The L1 cache is usually divided into Data (L1D) and Instruction (L1I) cache. Higher-level caches are often shared among the cores. Most CPUs use N-way set-associative caches, where each memory address maps to one of N *ways* within a specific *set*. In Linux environments, information on the cache hierarchy can be accessed via the `sysfs`.

C. The gem5 Simulator

gem5 [11] is an open-source Full System Simulator (FSS) used in computer architecture research, supporting different architectures like x86, ARM, and RISC-V. It provides different abstraction levels. In gem5's *atomic* mode, memory accesses are executed within a single transaction. The requester of the memory access is blocked until the access is completed. In the *timing* mode, gem5 provides detailed simulation of memory access timing, including latency, queuing delays, and bandwidth constraints. gem5 also offers different CPU models. The `SimpleCPU` does not model a pipeline, which makes it fast but not very accurate in terms of timing. It can be used in both, atomic and timing mode. gem5 further provides an `InOrderCPU` and `O3CPU` (out-of-order CPU), which are both equipped with a CPU pipeline model. In addition, gem5 distinguishes between the *full-system* mode and the *system call emulation* mode. The system call emulation mode aims to simulate user-space programs. It intercepts system calls of the target software and handles them on the host. The

```
class SimulatorRunner(Runner):
    def __init__(self, n_parallel=16, ...) :
        super(SimulatorRunner, self).__init__(...)
    def run(self, measure_inputs, build_results):
        # Build executables
        # Run executables in parallel on simulator
        return [AutoTVMRes0, AutoTVMRes1, ...]
```

Listing 3: Custom run function for AutoTVM flow

```
@tvm._ffi.register_func(<func_name>, override=True)
def local_run(inputs, build_results, ...) :
    # Build executables
    # Run executables in parallel on simulator
    return [AutoSchedRes0, AutoSchedRes1, ...]
```

Listing 4: Custom run function for Auto-Scheduler flow

system call emulation mode is faster than the full-system mode and focuses on software testing, as it does not simulate the Operating System (OS) and thus cannot capture any OS-specific behavior.

III. Implementation

In the first part of this chapter, we explain the implementation of the simulator interface. In the second part, we use this interface to connect TVM's autotuning with an x86, an ARM, and a RISC-V-based instruction-accurate simulator.

A. Simulator Interface

The simulator interface allows for executing autotuning implementations on user-level or syscall-emulation simulators instead of real hardware. This enables parallel execution of different implementations. TVM's autotuning requires a *builder* and a *runner*. The builder generates an object file that contains the compiled functionality of the workload. The runner executes the workload using the `tvm::runtime`.

For a simulator, a standalone executable is needed. The executable prepares the input tensors, allocates space for the output tensors, and calls the compiled workload. To integrate this behavior into AutoTVM, we implement a custom runner called `SimulatorRunner` by inheriting from the `Runner` class (see Listing 3). When the `run` function of the custom runner is called, it generates a main function, compiles it, and links it against the compiled object file. A function called `simulator_run` is called with the path to the executable. This function serves as a simulator interface and can be overwritten to use a simulator for execution. The return value needs to be a score quantifying the performance of the workload, e.g., the run time. A parameter named `n_parallel` defines how many simulators can be instantiated in parallel.

To integrate a simulator into the Auto-Scheduler, we override a function in TVM's function registry called `auto_scheduler.local_runner.run`. This can be seen in Listing 4. The return value is a list of `AutoSchedResults` containing scores. The remaining implementation works in the same way as for AutoTVM.

When generating an object file for a ML kernel, the kernel is optimized and lowered through TVM. LLVM is used for code generation. To produce cross-compiled object files, it is possible to specify an LLVM triple in TVM.

```
def conv2d(N,H,W,CO,CI,KH,KW,...)
    ifm=te.placeholder((N,CI,H,W),...)
    weights=te.placeholder((CO,CI,KH,KW),...)
    bias=te.placeholder((N,CO,1,1),...)
    conv=topi.nn.conv2d_nchw(ifm,weights,...)
    ofm=topi.nn.relu(conv+bias)
    # Return values: transferred as DLPack tensors
    return [ifm,kernel,bias,ofm]
```

Listing 5: Conv2D+Bias+ReLU kernel definition in TVM

Listing 5 shows the definition of a Conv2D+Bias+ReLU kernel as an example. The tensor shapes and parameters are passed as command line arguments to the executable.

B. gem5 Simulator Setup

To accurately predict timing, a cycle-accurate simulator is needed. However, cycle-accurate simulators often suffer from slow simulation speed. Additionally, obtaining cycle-accurate simulators with the required level of detail for commercial architectures is a challenge, as information about precise latencies associated with, e.g., memory components, buses, or microarchitecture details are rarely available open source.

We will demonstrate the feasibility of using a non-timing-accurate simulator for effective predictions based on quantitative parameters, without relying on timing information.

Our focus is on single-core workloads. To achieve fast simulation performance, we employ gem5 in *atomic* mode with the `SimpleCPU` model for x86, RISC-V, and ARM architectures (see Section II-C). gem5 allows to replicate the cache architecture while making it parameterizable to account for cache hits, misses, and replacements within the memory hierarchy (see Section II-B).

C. Score Predictor - Workflow and Notations

To use non-timing-accurate, i.e., instruction-accurate simulators, for score prediction, a predictor must be trained. Figure 4 illustrates the training and execution phases of this predictor. During the training phase, workloads are not only executed in gem5 but also natively on the target CPU. We require a distinct predictor for each architecture and *kernel type*. For instance, Conv2D+Bias+ReLU represents one specific kernel type. Note that the corresponding predictor can be applied to any combination of shapes and parameters of this kernel type. A fixed combination of shapes and parameters for a given kernel type is referred to as a *group*. The autotuning process generates various *implementations* (schedules) for each group. In the subsequent execution phase, we leverage the pre-trained predictor. The target CPU is **not** required anymore

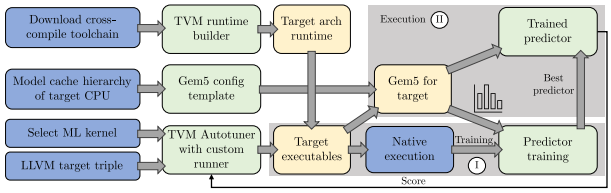


Fig. 4: Workflow of training (I) and execution (II) of a predictor for one target architecture and one kernel type

at this stage, which enables the simulation of architectures such as RISC-V on x86 platforms.

D. Score Predictor Training

The score predictor estimates a score S for an implementation I. The underlying principle is that these scores correlate with the run times, but only for different implementations of the same kernel type and group. Perfect prediction means $S_{I1} < S_{I2} \Rightarrow t_{I1} < t_{I2}$, where t_{Ix} is the run time of implementation x. Notably, comparisons between implementations across different kernel types or groups are not possible using the score. Given that timing details are unavailable, the score predictor relies solely on quantitative information rather than latencies. The relevant statistics derived from gem5 are:

- The number of the executed load/store/branch instructions divided by the total number of instructions.
- The total number of the executed instructions normalized to the total number of executed instructions of the group.
- Cache read/write replacements/hits/misses divided by read/write accesses of each cache.

Considering, e.g., L1D write ($L1Dw$) cache hits for implementation x, we determine:

$$P_{L1Dw}(I_x) = \frac{L1D_{w,hits}(I_x)}{L1D_{w,access}(I_x)} \tag{1}$$

Additionally, all parameters are also normalized to the group:

$$P_{L1Dw,norm}(I_x) = \frac{P_{L1Dw}(I_x) - \overline{P_{L1Dw}(\mathbf{I})}}{\overline{P_{L1Dw}(\mathbf{I})}} \tag{2}$$

The values $\overline{P_{L1Dw}(\mathbf{I})}$ represent the mean values across the group. We found that the most promising approach is to use these parameters as inputs for the predictor in both their original form $P_{L1Dw}(I_x)$ and their normalized form $P_{L1Dw,norm}(I_x)$. The output scores that are used for training are the measured run times normalized to the group, similar to Equation (2). Based on these inputs and the scores, we train different predictors, which can be used with different loss functions. In this work, we use Mean Squared Error (MSE), Mean Absolute Error (MAE), and Residual Sum of Squares (RSS). Below, a brief overview of the predictors is provided.

1) Multiple Linear Regression: MLR is an extension of linear regression. MLR is a simple predictor. It only models linear relationships. It finds a relation between multiple outputs y_k and multiple independent inputs x_i [20], in our case:

$$y = b_0 + b_1x_1 + b_2x_2 + ... + b_nx_n \tag{3}$$

2) Regression with DNNs: Regression using DNNs means predicting continuous outputs based on several input features. Typically, the networks consist of multiple fully connected layers, with the number of input neurons corresponding to the number of input features. Each hidden layer usually applies an activation function to introduce non-linearity and allow the network to capture complex relationships. The final layer has a number of neurons equal to the number of output variables. A common architecture might include a couple of hidden layers with decreasing numbers of neurons.

3) Bayesian Optimization: Bayesian optimization builds a surrogate model (in this case, a Gaussian process) that approximates the real/true objective function. It uses this model to predict which areas of the parameter space are likely to yield the best results and evaluates the function in those regions. An acquisition function balances the trade-off between exploring new regions of the space (exploration) and refining promising known regions (exploitation) [21]. Listing 6 shows the choice of the used objective function and loss function.

```
def objective_function(C, RBF_scale, noise):
    func = ConstantKernel(constant_value=C, ...) \
        * RBF(length_scale=RBF_scale, ...) \
        + WhiteKernel(noise_level=noise, ...)
    gp = GaussianProcessRegressor(func) \
        .fit(X_train, y_train)
    predictions = gp.predict(X_test)
    return - loss_function(y_test, predictions)
```

Listing 6: Objective function for bayes optimization

The hyperparameters of the Gaussian process are the input parameters of the `objective_function`. The Bayesian optimization framework maximizes the objective function through a series of iterations. Each iteration involves fitting a Gaussian process model with new hyperparameters and updating the probabilistic model based on the results.

4) XGBoost: Extreme Gradient Boosting (XGBoost [22]) is a powerful algorithm for regression tasks. It uses an ensemble of decision trees to predict continuous outcomes. In XGBoost, trees are built sequentially, with each tree trying to minimize the errors made by the previous ones by adjusting residuals. The algorithm optimizes a loss function using gradient descent. The hyperparameters of the XGBoost algorithm are, e.g., the learning rate, max. tree depth, and regularization parameters.

E. Score Predictor Inference

For inference, the trained predictors are integrated into the execution pipeline, as shown in Figure 4. A challenge arises because input parameters, such as $P_{L1Dw,norm}(I_x)$, cannot be determined for new, unknown groups. This limitation is due to the Auto-Scheduler generating implementations batch-wise based on prior scores (see Section II-A), preventing the computation of mean values like $\overline{P_{L1Dw}(\mathbf{I})}$ at the beginning.

To address this, we allow approximating mean values using two approaches: *static* and *dynamic* windows. In the static approach, mean values are calculated from the first $\overline{w}$ samples. In the dynamic window approach, mean values are adaptively adjusted over time. The batch size, and thus the window size $\overline{w}$, is typically large enough that no accuracy loss compared to using $\overline{P_{L1Dw}(\mathbf{I})}$ was observed in the experiments.

IV. Results

To verify the quality of our predictors, all benchmarks are executed on three different CPU architectures: x86, ARM, and RISC-V. For x86, we use a 64 bit 2.2 GHz AMD Ryzen 7 5800X 8-Core processor; for ARM, we use a 64 bit 1.5 GHz Raspberry Pi 4 Model B with an ARM Cortex-A72 processor; and for RISC-V, we use a 64 bit 1.2 GHz SiFive U74-MC processor.

TABLE I: Cache sizes and hierarchy of the used CPUs

	L1 Data			L1 Instruction			L2			LLC (L3)		
	size	sets	assoc	size	sets	assoc	size	sets	assoc	size	sets	assoc
x86	32K	64	8	32K	64	8	512K	1024	8	32768K	32768	16
ARM	32K	256	2	48K	256	3	1024K	1024	16	-	-	-
RISC-V	32K	64	8	32K	64	8	2048K	2048	16	-	-	-

TABLE II: Shapes of the used Conv2D+Bias+ReLU kernels

group	N	H	W	CO	CI	KH	KW	stride	pad
0	1	224	224	64	3	7	7	(2,2)	(3,3)
1	1	56	56	64	64	3	3	(1,1)	(1,1)
2	1	56	56	128	64	3	3	(2,2)	(1,1)
3	1	28	28	256	128	3	3	(2,2)	(1,1)
4	1	14	24	512	256	3	3	(2,2)	(1,1)

Table I lists the cache hierarchies of these CPUs. We model them in gem5 (see Section III-B). All cache line sizes are 64 B. The ARM and RISC-V CPUs feature a shared L2 cache but no L3 cache. All gem5 simulations are executed on the x86 machine. We use five groups of Conv2D+Bias+ReLU kernels from a ResNet [23] architecture as benchmarks. Table II lists the shapes and parameters of these groups. For training the predictors, the Auto-Scheduler generates 500 implementations per group, with 100 implementations used for the test set.

To determine the reference execution time t_{ref}, each implementation is executed $N_{exe} = 15$ times, with the median value taken as the reference. Additionally, cooldown times of $t_{cooldown} = 1s$ are inserted between each run to ensure more reproducible measurements. Workloads are not executed in parallel on real hardware, as this could affect the measurements. This means that execution on K parallel simulators is faster than native (sequential) execution on a single device.

$$K = \left\lceil \frac{t_{simulator}}{(t_{cooldown} + t_{ref}) \cdot N_{exe}} \right\rceil \tag{4}$$

Our measurement setup ($N_{exe} = 15$, $t_{cooldown} = 1s$) results in $K_{x86} \in [7, 97]$, $K_{ARM} \in [4, 31]$, and $K_{RISC-V} \in [3, 21]$ for the tested workloads. This means that in the best case, *only 3 parallel executions on the x86 machine are sufficient to achieve a speedup over native execution on the RISC-V CPU.*

A. Prediction of Non-Trained Groups

First, we demonstrate that a trained predictor can be utilized even when a specific kernel group was not included in the training data. This capability is crucial as we aim to develop one predictor per architecture and kernel type (e.g., Conv2D+Bias+ReLU) that remains valid across all groups with different shapes and parameters (see Section III-C).

To achieve this, we initially train Bayesian predictors for all architectures using all groups. Subsequently, we train additional predictors using only groups 0, 1, 2, 4, and 5. Figure 5 compares the test set of group 3 when group 3 is included in the training (Figures 5a to 5c) against using the same samples when group 3 is not included in the training (Figures 5d to 5f). The x-axis represents the individual samples. The reference time t_{ref} shows the median values of the measured, in ascending order sorted run times of the implementations. To obtain the predicted run time t_{pred}, the predicted scores are sorted in ascending order, with the corresponding measured run time plotted according to these scores. A perfect prediction would mean that $t_{pred} == t_{ref}$.

Although not all scores are perfectly ordered, a clear ascending trend is evident. Predictions for ARM and RISC-V architectures appear more accurate than those for x86. This discrepancy may arise from fewer HW optimizations on these embedded CPUs. Furthermore, each predictor is only as good as its reference measurements. Since execution times on x86 are significantly faster, reference measurements show greater variability compared to longer run times on the embedded architectures. Overall, visual inspections reveal no clear advantage between included and non-included training groups. *This demonstrates that the predictor can effectively be used even for groups that are not present in the training data.* To enable better comparisons of predictions, we will introduce three different evaluation metrics in the following.

B. Evaluation Metrics for the Predictors

The most important aspect is predicting the fastest run time. Therefore, we introduce E_{top1} and the rank metric R_{top1}. E_{top1} represents the relative error between the run times of the fastest reference measurement and the sample with the best predicted score:

$$E_{top1} \coloneqq \left(1 - \frac{t_{ref}[0]}{t_{pred}[0]}\right) \cdot 100\% \tag{5}$$

Rank R_{top1} indicates the relative position at which the fastest sample was ranked by the predictor:

$$R_{top1} \coloneqq \frac{100\%}{|t_{ref}|} \cdot \left(\underset{x}{\operatorname{argmin}} \left(t_{pred}[x] == t_{ref}[0]\right) + 1\right) \tag{6}$$

For example, $R_{top1} = 3\,\%$ means that the fastest sample was ranked within the top 3 % of predictions.

Additionally, we need a metric to evaluate sorting quality. Consecutive non-monotonically increasing samples should be penalized, as well as their extent of deviation. Therefore, we introduce the Quality Score Q:

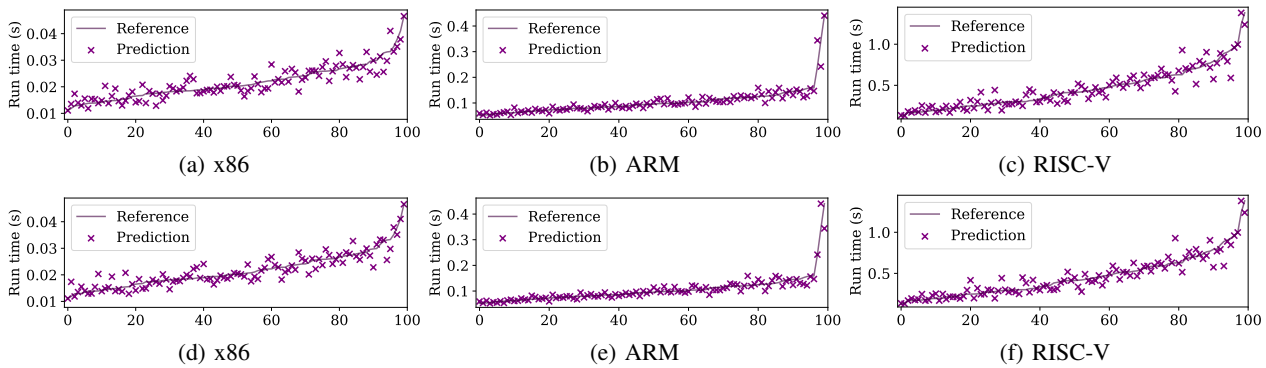
$$Q \coloneqq \frac{100\%}{|t_{ref}|} \cdot \sum_i \frac{t_{ref}[i] - min(t_{ref}[i], t_{ref}[i+1])}{t_{ref}[i]} \tag{7}$$

To prevent deviations in the slower part of the run times from being disproportionately weighted, we evaluate Q separately for the lower 50 % of the run times (Q_{low}) and the higher 50 % of the run times (Q_{high}). For all metrics, a smaller value indicates better performance.

C. Comparison of Predictors

Next, we compare the different predictors (see Section III-D). We tested various loss functions, activation functions, and parameters. Given that the XGBoost algorithm has many hyperparameters, we employed grid search [24] for tuning. The used configurations (after tuning) are:

- **Linear Regression**: RSS loss
- **DNN**: 6 dense layers (number of neurons: 128, 128, 64, 32, 16, 1), linear output activation, tanh hidden layer activation, MAE loss, adam optimizer



(a) x86 (b) ARM (c) RISC-V

(d) x86 (e) ARM (f) RISC-V

Fig. 5: Sorted run time predictions for the test set of group 3 (a)-(c) when group 3 is **included** in the training vs. the same samples of group 3 (d)-(f) when group 3 is **not included** in the training

TABLE III: Prediction results for x86-based CPU

ID	LinReg				DNN				Bayes				XGBoost			
	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)
0	10.7	3.0	3.0	8.5	1.4	2.7	2.2	3.0	12.5	3.0	2.7	3.0	7.0	2.7	2.4	2.0
1	11.2	3.6	3.5	3.0	3.2	2.8	2.8	2.0	8.9	2.6	2.8	3.5	1.5	2.7	2.4	2.5
2	15.0	3.2	3.2	3.0	7.6	2.7	2.3	2.0	0.0	2.6	2.3	1.0	2.2	2.4	2.2	2.0
3	8.8	3.2	3.0	2.0	0.7	2.4	2.4	1.5	0.7	2.7	2.7	1.5	0.0	2.8	2.4	1.0
4	0.0	3.4	3.3	1.0	2.8	3.1	2.2	3.0	5.4	2.5	2.6	2.0	2.0	2.8	2.4	2.0

- **Bayes**: Objective function shown in Listing 6, MSE loss
- **XGBoost**: Column subsample ratio 0.6, learning rate 0.05, max. tree depth 3, alpha 0, lambda 0.1, gradient boost trees 300, min. child weight 1, subsample ratio for training 0.8, MSE loss

For the evaluation, all groups were included in the training. One predictor (i.e., one of LinReg, DNN, etc.) was trained per CPU architecture. Each training was performed 10 times with a random selection of training and test sets. Scores were subsequently calculated based on the median predictions from the test sets. Table III shows the results for x86, Table IV for ARM, and Table V for RISC-V.

For the x86 architecture, the DNN, Bayesian optimization, and XGBoost yield good scores. XGBoost achieves the best average R_{top1} score, with a maximum of 2.5 %. XGBoost also had the smallest prediction error, with an average of $E_{top1} = 2.54\,\%$. For ARM and RISC-V, the scores are slightly better. For ARM, DNNs, Bayesian optimization, and XGBoost achieve a R_{top1} score smaller or equals 2.5 %. All E_{top1} errors are below 5 %, and often even below 2 %. Bayesian optimization correctly predicts the optimum in three out of five cases and demonstrates the lowest Q_{low} and Q_{high}. For RISC-V, even the linear model performs quite well, with exceptions in E_{top1} for group 0 and 4. For other predictors, the optimum is often included in the top 2 % of predictions. The DNN provides the best results with $E_{top1} \leq 3.6\,\%$ and $R_{top1} \leq 2\,\%$.

In summary, it is possible to train predictors for forecasting. *For RISC-V and ARM, a prediction error E_{top1} of less than 5 % is achievable.* If the goal is to find the best sample, it is *sufficient to re-execute the top 2 %-3 % of the predictions* later on a real architecture.

TABLE IV: Prediction results for ARM-based CPU

ID	LinReg				DNN				Bayes				XGBoost			
	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)
0	9.9	3.4	2.9	3.5	0.2	2.8	2.3	1.5	0.0	2.6	2.0	1.0	2.7	3.0	2.2	1.5
1	6.4	3.6	3.2	3.0	4.6	3.2	2.6	2.0	4.6	3.4	2.4	2.0	4.3	3.2	2.6	2.5
2	7.7	3.6	2.3	5.0	0.7	3.3	2.4	1.5	4.3	3.2	2.3	2.5	0.3	3.4	2.2	1.5
3	7.7	2.8	2.6	2.5	1.1	2.8	2.3	1.5	0.0	2.8	2.0	1.0	4.0	3.2	2.2	2.0
4	2.2	3.5	2.5	4.0	0.2	3.1	2.6	1.5	0.0	2.6	2.2	1.0	1.0	3.0	2.3	2.0

TABLE V: Prediction results for RISC-V-based CPU

ID	LinReg				DNN				Bayes				XGBoost			
	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)	E_{top1}(%)	Q_{low}(%)	Q_{high}(%)	R_{top1}(%)
0	10.9	4.0	3.9	4.0	2.2	4.0	4.0	2.0	0.0	3.8	4.2	1.0	0.0	3.4	4.1	1.0
1	0.0	4.0	4.3	1.0	0.6	3.7	4.5	1.5	2.5	3.4	4.4	2.0	4.6	3.5	4.5	2.5
2	0.0	3.4	3.7	1.0	0.0	3.2	3.8	1.0	0.0	2.8	3.4	1.0	0.0	3.0	3.8	1.0
3	0.0	3.6	3.8	1.0	0.0	3.2	3.9	1.0	2.0	2.8	3.7	1.5	4.4	3.0	3.6	2.0
4	11.0	4.0	4.2	3.0	3.6	3.6	4.2	1.5	10.7	3.2	4.0	2.0	8.2	3.8	4.0	3.0

V. Conclusion and Future Work

In this work, we introduced an interface for executing autotuning workloads on simulators and explored the feasibility of using instruction-accurate simulators for autotuning of ML workloads. We trained and compared different score predictors for the x86, ARM, and RISC-V architectures. Our results show that the tuned predictors can identify optimal implementations within the top 3 % of predictions. In the case of limited availability of the target HW, our approach can even be used to accelerate autotuning. In the best case, 3 parallel simulations on the used x86 machine were sufficient to replace one RISC-V board. Our research lays the groundwork for using instruction-accurate simulations for performance estimation.

Future work will focus on benchmarking a broader range of CPUs to train more generalized predictors. These generalized predictors can then be applied to previously untested CPUs, enhancing our methodology's appeal for pre-silicon software development.

REFERENCES

[1] K. Datta *et al.*, "Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures," in *SC'08: Proceedings of the 2008 ACM/IEEE conference on Supercomputing*. IEEE, 2008, pp. 1–12.

[2] L. Lin and M. Gen, "Auto-tuning strategy for evolutionary algorithms: balancing between exploration and exploitation," *Soft Computing*, vol. 13, pp. 157–168, 2009.

[3] T. Chen *et al.*, "TVM: An automated End-to-End optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594.

[4] L. Zheng *et al.*, "Ansor: Generating High-Performance tensor programs for deep learning," in *14th USENIX symposium on operating systems design and implementation (OSDI 20)*, 2020, pp. 863–879.

[5] Y.-S. Hsieh and Y.-P. You, "DLOOPT: An Optimization Assistant on AutoTVM for Deep Learning Operators," *Journal of Signal Processing Systems*, vol. 95, no. 5, pp. 585–607, 2023.

[6] W. Grunewald and T. Ungerer, "Towards extremely fast context switching in a block-multithreaded processor," in *Proceedings of EUROMICRO 96. 22nd Euromicro Conference. Beyond 2000: Hardware and Software Design Strategies*. IEEE, 1996, pp. 592–599.

[7] S. Zhuravlev, J. C. Saez, S. Blagodurov, A. Fedorova, and M. Prieto, "Survey of scheduling techniques for addressing shared resources in multicore processors," *ACM Computing Surveys (CSUR)*, vol. 45, no. 1, pp. 1–28, 2012.

[8] T. Benoit-Cattin, D. Velasco-Montero, and J. Fernández-Berni, "Impact of thermal throttling on long-term visual inference in a CPU-based edge device," *Electronics*, vol. 9, no. 12, p. 2106, 2020.

[9] D. Brodowski, N. Golde, R. J. Wysocki, and V. Kumar, "CPU frequency and voltage scaling code in the Linux (TM) kernel," *Linux kernel documentation*, vol. 66, 2013.

[10] F. Bellard, "QEMU, a fast and portable dynamic translator." in *USENIX annual technical conference, FREENIX Track*, vol. 41, no. 46. California, USA, 2005, pp. 10–5555.

[11] N. Binkert *et al.*, "The gem5 simulator," *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011.

[12] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to linear regression analysis*. John Wiley & Sons, 2021.

[13] J. Du and Y. Xu, "Hierarchical deep neural network for multivariate regression," *Pattern Recognition*, vol. 63, pp. 149–157, 2017.

[14] P. I. Frazier, "Bayesian optimization," in *Recent advances in optimization and modeling of contemporary problems*. Informs, 2018, pp. 255–278.

[15] T. Chen *et al.*, "Xgboost: extreme gradient boosting," *R package version 0.4-2*, vol. 1, no. 4, pp. 1–4, 2015.

[16] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," *Acm Sigplan Notices*, vol. 48, no. 6, pp. 519–530, 2013.

[17] V. Kelefouras and G. Keramidas, "Design and implementation of deep learning 2D convolutions on modern CPUs," *IEEE Transactions on Parallel and Distributed Systems*, 2023.

[18] R. Li, Y. Xu, A. Sukumaran-Rajam, A. Rountev, and P. Sadayappan, "Analytical characterization and design space exploration for optimization of CNNs," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 928–942.

[19] S. Kumar and P. Singh, "An overview of modern cache memory and performance analysis of replacement policies," in *2016 IEEE International Conference on Engineering and Technology (ICETECH)*. IEEE, 2016, pp. 210–214.

[20] G. K. Uyanık and N. Güler, "A study on multiple linear regression analysis," *Procedia-Social and Behavioral Sciences*, vol. 106, pp. 234–240, 2013.

[21] P. I. Frazier, "A tutorial on Bayesian optimization," *arXiv preprint arXiv:1807.02811*, 2018.

[22] T. Chen, T. He, M. Benesty, and V. Khotilovich, "Package 'xgboost'," *R version*, vol. 90, no. 1-66, p. 40, 2019.

[23] Z. Wu, C. Shen, and A. Van Den Hengel, "Wider or deeper: Revisiting the ResNet model for visual recognition," *Pattern recognition*, vol. 90, pp. 119–133, 2019.

[24] S. Putatunda and K. Rama, "A comparative analysis of hyperopt as against other approaches for hyper-parameter optimization of XGBoost," in *Proceedings of the 2018 international conference on signal processing and machine learning*, 2018, pp. 6–10.